claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_095035a0-e9f\agent-a1c9968fa8377a1f2.jsonl`
- SHA-256: `4916fd93e545c0b2ed4ee92cedfe946a35359bc9003a73350254a114ddeea4bd`
- Source modified: `2026-06-13T19:17:02+00:00`
- Imported at: `2026-07-08T16:00:00+00:00`
- project: `wf_095035a0-e9f`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`

Transcript

1. user (2026-06-13T19:13:53.220Z)

Adversarially VERIFY this fusion-P2 review finding by reading the actual code. Try to REFUTE it; only confirm real=true if it genuinely holds against the code. Default to real=false if uncertain or if it's a nitpick/false-positive.

FINDING: {"title":"All six verification targets PASS ? fusion P2 integration is sound; no high/med integration bugs found","severity":"info","file":"backend/config/models.py:78-108, backend/pipeline/segmenters/__init__.py:7,17, backend/pipeline/segmenters/fusion_hybrid.py:130","detail":"Verified each requested item against the actual code + live Python (pydantic present, torch/cv2 present in this env): (1) DEFAULTS REPRODUCE CONTROL: FusionConfig() => substrate='wasb', cue_flags={}, min_crossings=0, min_players=0, court_polygon=None, velocity_threshold=0.02 ? all no-ops; _select_for_handoff returns range(len(windows)) and _enrich_candidate_stats adds only net_crossings (no person keys) when person OFF. (2) PYDANTIC TYPES VALID: FusionConfig instantiates and validates; Literal['wasb','tracknet'] and Literal['x','y'] reject bad values; Optional[list[list[float]]] parses [[0.1,0.1],...]; dict[str,bool] cue_flags works; min_crossings/min_players ge=0; net_position ge=0/le=1. (3) DICT-SHIM EXPOSES FUSION: AppConfig.get('indexing') returns model_dump(mode='json') => a fully nested plain dict with idx_cfg['fusion'] itself a dict, so idx_cfg.get('fusion',{}).get('substrate'/'cue_flags'/'min_crossings'/'court_polygon') all resolve correctly. main.py:190/737 constructs segmenter_cls(db, global_config) with an AppConfig instance, so the shim path is the real production path and works end-to-end. (4) UNKNOWN FUSION KEYS DON'T CRASH: FusionConfig has no extra='allow' so default extra='ignore' applies ? unknown keys are silently dropped (matching the WASB-section pattern), no ValidationError, both directly and via AppConfig(indexing={'fusion':{'unknown_key':123}}). (5) NO NAME CLASH: registry lists motion/gemini/yolo_hybrid/tracknet_hybrid/wasb_hybrid/fusion_hybrid ? fusion_hybrid is distinct; family='fusion' is a fresh config section that does not collide with wasb/tracknet. (6) REGISTRY PROBE SAFE: FusionSegmenter(None,{}) succeeds ? __init__ calls super().__init__ (sets self.config={}, self._person=None) and .name is a plain string property; description also resolves. Test suites confirm: tests/test_fusion_hybrid.py, tests/test_fusion_features.py, tests/test_config.py, tests/test_trajectory_hybrid_equivalence.py all pass (78 passed in the segmenter/registry/fusion/config/person slice). The trajectory_hybrid refactor preserves the twins byte-identical (base _enrich_candidate_stats returns exactly the 4 motion keys; handoff loop indexes candidate_ids[wi]/all_candidate_windows[wi] with full-list indices from handoff_indices ? no off-by-one). No integration bug rising to high or medium.","fix":"No action required for correctness. The low-severity polish items below are optional."}

Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer. The change implements fusion P2 (MULTI_SIGNAL_FUSION_PLAN.md): a new default-OFF `FusionSegmenter` (backend/pipeline/segmenters/fusion_hybrid.py) that subclasses TrajectoryHybridSegmenter, persists shuttle net_crossings (Gate A) + optional person-cue features into candidate stats, and optionally gates the AI handoff. Pure feature math is in backend/pipeline/detectors/fusion_features.py. NEW files: backend/pipeline/detectors/fusion_features.py, backend/pipeline/segmenters/fusion_hybrid.py. EDITED files (use `git --no-pager diff -- <path>` to see edits): backend/pipeline/segmenters/trajectory_hybrid.py, backend/pipeline/detectors/person_detector.py,

backend/config/models.py, backend/pipeline/segmenters/__init__.py. READ the actual files. The work is uncommitted in the working tree on branch master. Quote file:line. Only report REAL issues; rank by severity (high/med/low).

2. assistant (2026-06-13T19:13:56.395Z)

I'll verify this finding adversarially by reading the actual code. The finding claims all six verification targets PASS with no integration bugs. Let me check each claim.

3. user (2026-06-13T19:13:57.950Z)

```
warning: in the working copy of 'backend/pipeline/segmenters/trajectory_hybrid.py', LF will be
replaced by CRLF the next time Git touches it
warning: in the working copy of 'tests/test_trajectory_hybrid_equivalence.py', LF will be
replaced by CRLF the next time Git touches it
 backend/config/models.py                          | 34 ++++++++++++++++
 backend/pipeline/detectors/person_detector.py     | 47 ++++++++++++++++++++++
 backend/pipeline/segmenters/__init__.py           |  2 +
 backend/pipeline/segmenters/trajectory_hybrid.py  | 50 ++++++++++++++++++++++-----
 docs/README.md                                    |  3 +-
 tests/test_config.py                              | 30 ++++++++++++++
 tests/test_trajectory_hybrid_equivalence.py       | 22 ++++++++++
 7 files changed, 177 insertions(+), 11 deletions(-)
---STATUS---
 M backend/config/models.py
 M backend/pipeline/detectors/person_detector.py
 M backend/pipeline/segmenters/__init__.py
 M backend/pipeline/segmenters/trajectory_hybrid.py
 M docs/README.md
 M tests/test_config.py
 M tests/test_trajectory_hybrid_equivalence.py
?? artifacts/
?? backend/pipeline/detectors/fusion_features.py
?? backend/pipeline/segmenters/fusion_hybrid.py
?? docs/QUALITY_ITERATIONS.md
?? scratch/HIGH_RISK_REMEDIATION_PLAN.md
?? scratch/PR1_BODY.md
?? scratch/PR1_HANDOFF.md
?? scratch/PR2_HANDOFF.md
?? scratch/PR2_deferred_hardening_status.patch
?? scratch/PR88_FIX.md
?? scratch/arial.ttf
?? scratch/debug_wsl.py
?? scratch/keep_awake.ps1
?? scratch/probe_cv2_count.py
?? scratch/probe_gemini.py
?? scratch/repro_drawtext_font.py
?? scratch/repro_movie_overlay.py
?? scratch/run_gx010125.py
?? scratch/stitch_min5.py
?? scratch/trim_video.py
?? tests/test_fusion_features.py
?? tests/test_fusion_hybrid.py
```

4. assistant (2026-06-13T19:14:00.978Z)

The finding references specific files. Note the diff stat shows `backend/config/models.py` with 34 lines added, not exactly matching "lines 78-108" but let me read the actual files. Let me read the key files in parallel.

5. user (2026-06-13T19:14:02.395Z)

```
1       """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
```

```
    Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
    TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
    do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
    shuttle-seeded window via two overridable seams the engine exposes:

    - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
      GPU-free, computed from the trajectory we already have); and, only when the person cue
      is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
      (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
      fusion (P3). The person path lazily builds ONE `PersonDetector` ? so the default path
      never imports torch / touches the GPU.
    - ``_select_for_handoff`` ? optional served Gate A/B: when ``indexing.fusion.min_crossings``
      (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows are
      dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
      **Persisted candidates remain the full superset** regardless, so the offline experiment
      harness can re-score any variant.

    Defaults reproduce the substrate exactly: net_crossings/person features are persisted but
    nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config seeds
    and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the default
    segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift through the
    experiment harness (EXPERIMENT_HARNESS.md), never silently.
    """
    from typing import Any, Dict, List, Optional, Tuple

    from backend.pipeline.segmenters.base import segmenter_registry
    from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
    from backend.pipeline.detectors.base import DetectorRunner
    from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
    from backend.pipeline.detectors import rally_gate
    from backend.pipeline.detectors import fusion_features as ff


    class FusionSegmenter(TrajectoryHybridSegmenter):
        """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate B)."""

        family = "fusion"
        display_label = "Fusion"

        def __init__(self, db: Any, config: Any):
            super().__init__(db, config)
            # Built lazily only if the person cue is enabled (keeps torch/GPU off the default path).
            self._person: Optional[Any] = None

        @property
        def name(self) -> str:
            return "fusion_hybrid"

        @property
        def description(self) -> str:
            return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate rallies, "
                    "net-crossing Gate A + optional person-occupancy Gate B adjudicate them, and the "
                    "AI provider sets semantic boundaries.")
```

```

        def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
            substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
            if substrate == "tracknet":
                cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
                return TrackNetRunner(cfg), {
                    "weights_path": cfg.weights_path,
                    "queue_length": cfg.queue_length,
                    "fusion_substrate": "tracknet",
                }
            wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
            return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}

        def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
                                    frame_width: float, video_path: str,
                                    idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
            stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
            # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
            # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
            # over the whole trajectory by rally_gate (camera-agnostic).
            gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
min_crossings=0)
            stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0

            fcfg = idx_cfg.get("fusion", {})
            if fcfg.get("cue_flags", {}).get("person", False):
                stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
            return stats

        def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
                             frame_width: float, video_path: str,
                             fcfg: Dict[str, Any]) -> Dict[str, float]:
            """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
            frame range and compute the scale/translation-invariant person features. Lazily
builds
            ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
            # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
            from backend.pipeline.detectors.person_detector import PersonDetector
            if self._person is None:
                self._person = PersonDetector(self.config)

            frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
            start_f, end_f = int(cw["start"] * fps), int(cw["end"] * fps)
            det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
            fw = det["frame_width"] or frame_width
            fh = det["frame_height"] or 0.0

            poly = fcfg.get("court_polygon")
            polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
            net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
            shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
            return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
                                              court_polygon=polygon, net=net)

        def _select_for_handoff(self, windows: List[Dict[str, Any]],
                                window_stats: List[Dict[str, Any]],
```

```
111                                 idx_cfg: Dict[str, Any]) -> List[int]:
112             """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
113             (0) keep every window ? identical to the substrate. Persisted candidates are
114             untouched (full superset)."""
115             fcfg = idx_cfg.get("fusion", {})
116             min_crossings = int(fcfg.get("min_crossings", 0))
117             min_players = int(fcfg.get("min_players", 0))
118             person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
119
120             keep: List[int] = []
121             for i, st in enumerate(window_stats):
122                 if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
123                     continue
124                 if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
125                     continue
126                 keep.append(i)
127             return keep
128
129
130     segmenter_registry.register(FusionSegmenter)
131
```

6. user (2026-06-13T19:14:02.722Z)

```
1       """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
2
3       NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4       math is unit-testable in a GPU-free dev container. The detection (person boxes,
5       shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6       module only turns those signals into a handful of **scale/translation-invariant**
7       features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
8       memorise this court/camera and fail to generalise off a small labelled set).
9
10      Conventions:
11      - A person box is ``(x1, y1, x2, y2)`` in **pixels**; ``(track_id, box)`` when motion
needs it.
12      - ``per_frame`` is ``{frame_idx: [(track_id, box), ...]}`` keyed by the ORIGINAL video
frame
13        index (so it aligns directly with the shuttle trajectory's ``.frame`` ? no clip re-
timing).
14      - A shuttle point has ``.frame``, ``.visible``, ``.x``, ``.y`` (pixels) ? the WASB
trajectory shape.
15      - ``court_polygon`` is normalised 0-1 ``[(x, y), ...]`` or ``None`` (None ? no mask,
every
16        detection counts ? the player-count-only mode).
17      - ``net`` is ``(axis 'x'|'y', position 0-1 normalised)`` or ``None`` (None ? two-sided =
0.0).
18
19      Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20      floats in [0, 1] except ``mean_player_motion`` (a non-negative normalised speed).
21      """
22      from __future__ import annotations
23
24      from typing import Dict, List, Optional, Sequence, Tuple
25
26      BBox = Tuple[float, float, float, float]
27      Net = Tuple[str, float]
28
29
30      def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
31          """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
```

```
        (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
        if polygon is None or len(polygon) < 3:
            return False
        x, y = point
        inside = False
        n = len(polygon)
        j = n - 1
        for i in range(n):
            xi, yi = polygon[i]
            xj, yj = polygon[j]
            # Does the horizontal ray at y cross edge (i, j)?
            if (yi > y) != (yj > y):
                x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
                if x < x_cross:
                    inside = not inside
            j = i
        return inside


    def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
        """Normalised foot point (bottom-centre) of a person box ? where the player stands,
        the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
        if frame_width <= 0 or frame_height <= 0:
            return (0.0, 0.0)
        x1, _y1, x2, y2 = box
        return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)


    def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
        if frame_width <= 0 or frame_height <= 0:
            return (0.0, 0.0)
        x1, y1, x2, y2 = box
        return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)


    def person_in_court(box: BBox, frame_width: float, frame_height: float,
                        court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
        """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
        True (count everyone) when it is None."""
        if court_polygon is None:
            return True
        return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)


    def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
                        frame_height: float,
                        court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
List[int]:
        """Per-frame count of in-court persons (one entry per sampled frame)."""
        return [
            sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
court_polygon))
            for _frame_idx, dets in sorted(per_frame.items())
        ]


    def _median(vals: Sequence[float]) -> float:
        if not vals:
            return 0.0
        s = sorted(vals)
        n = len(s)
        return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))

```

```
93
94      def players_in_court(counts: Sequence[int]) -> float:
95          """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
96          return _median(list(counts))
97
98
99      def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
100         """Fraction of sampled frames with >= `min_players` in court (track stability vs
flicker)."""
101         if not counts:
102             return 0.0
103         return sum(1 for c in counts if c >= min_players) / len(counts)
104
105
106     def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
107                            frame_width: float, frame_height: float) -> float:
108         """Mean per-track centre displacement between consecutive sampled frames, normalised
109         by the frame width (resolution-invariant). 0.0 if <2 frames or no shared tracks."""
110         if frame_width <= 0 or len(per_frame) < 2:
111             return 0.0
112         frames = sorted(per_frame.keys())
113         steps: List[float] = []
114         for a, b in zip(frames, frames[1:]):
115             prev = {tid: box for tid, box in per_frame[a]}
116             for tid, box in per_frame[b]:
117                 if tid in prev:
118                     cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
119                     cx1, cy1 = _center_norm(box, frame_width, frame_height)
120                     steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
121         if not steps:
122             return 0.0
123         return sum(steps) / len(steps)
124
125
126     def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
127                          frame_height: float, net: Optional[Net],
128                          court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
129         """Fraction of sampled frames with >=1 in-court person on BOTH net halves.
130
131         A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
132         The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
0.0.
133         """
134         if net is None or not per_frame:
135             return 0.0
136         axis, pos = net
137         both = 0
138         for _frame_idx, dets in per_frame.items():
139             near, far = False, False
140             for _tid, box in dets:
141                 if not person_in_court(box, frame_width, frame_height, court_polygon):
142                     continue
143                 fx, fy = _foot_norm(box, frame_width, frame_height)
144                 coord = fx if axis == "x" else fy
145                 if coord < pos:
146                     near = True
147                 else:
148                     far = True
149             if near and far:
150                 both += 1
151         return both / len(per_frame)
152
153
154     def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
float,
```

```
                        pad_frac: float) -> bool:
        """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
        width/height (the 'adjacent / in-hand' tolerance)?"""
        x1, y1, x2, y2 = box
        px = pad_frac * frame_width
        py = pad_frac * frame_height
        return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)


    def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
                                  frame_width: float, frame_height: float,
                                  pad_frac: float = 0.03) -> float:
        """Fraction of (visible shuttle frames that were also person-sampled) in which the
        shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
~0**
        ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
        """
        if not per_frame or frame_width <= 0 or frame_height <= 0:
            return 0.0
        aligned = 0
        colocated = 0
        for p in shuttle_points or []:
            if not getattr(p, "visible", False):
                continue
            dets = per_frame.get(int(p.frame))
            if dets is None:          # shuttle frame wasn't person-sampled ? can't judge
                continue
            aligned += 1
            if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
                   for _tid, box in dets):
                colocated += 1
        return colocated / aligned if aligned else 0.0


    # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
    # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
    PERSON_FEATURE_NAMES = [
        "players_in_court", "two_sided_motion", "mean_player_motion",
        "in_court_ratio", "shuttle_player_colocation",
    ]


    def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
                                frame_width: float, frame_height: float,
                                court_polygon: Optional[Sequence[Tuple[float, float]]] =
None,
                                net: Optional[Net] = None,
                                min_players: int = 2) -> Dict[str, float]:
        """Aggregate the per-window person features from per-frame detections + the in-
window
        shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
        counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
        return {
            "players_in_court": players_in_court(counts),
            "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
court_polygon),
            "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
            "in_court_ratio": in_court_ratio(counts, min_players),
            "shuttle_player_colocation": shuttle_player_colocation(
                per_frame, shuttle_points, frame_width, frame_height),
        }

```

7. assistant (2026-06-13T19:14:05.501Z)

Now let me read the trajectory_hybrid.py diff (the refactor that exposes the seams), the config models, and the segmenters __init__.

8. user (2026-06-13T19:14:06.716Z)

```
warning: in the working copy of 'backend/pipeline/segmenters/trajectory_hybrid.py', LF will be
replaced by CRLF the next time Git touches it
diff --git a/backend/pipeline/segmenters/trajectory_hybrid.py
b/backend/pipeline/segmenters/trajectory_hybrid.py
index b2847c9..149282e 100644
--- a/backend/pipeline/segmenters/trajectory_hybrid.py
+++ b/backend/pipeline/segmenters/trajectory_hybrid.py
@@ -78,6 +78,31 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
         except (ValueError, TypeError):
             return max(3, int(duration / 0.8))

+    # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged) -------
+    def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
+                                frame_width: float, video_path: str,
+                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
+        """Stats dict persisted into ``candidate_segments.stats`` for one window. The BASE
+        returns exactly the 4 motion keys, so the trajectory twins persist byte-identical
+        rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
+        features). ``points`` are the whole-video trajectory points (TrajectoryPoint)."""
+        return {
+            "peak_velocity": cw["peak_velocity"],
+            "mean_velocity": cw["mean_velocity"],
+            "active_frame_count": cw["active_frame_count"],
+            "track_id_count": cw["track_id_count"],
+        }
+
+    def _select_for_handoff(self, windows: List[Dict[str, Any]],
+                            window_stats: List[Dict[str, Any]],
+                            idx_cfg: Dict[str, Any]) -> List[int]:
+        """Indices of the candidate windows that proceed to the AI handoff (and thus may
+        become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
+        ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
+        Persisted candidates are NOT affected ? they remain the full superset for offline
+        re-scoring."""
+        return list(range(len(windows)))
+
     def process_video(self, video_id: str, video_path: str,  # type: ignore[override]
                       player_pool: Optional[List[str]] = None,
                       max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
@@ -172,31 +197,36 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
                             f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
                             f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")

+        # Per-window stats via the enrichment hook ? base = the 4 motion keys; the fusion
+        # segmenter adds net_crossings + person-cue features. Computed once, reused for both
+        # persistence and the handoff gate below.
+        window_stats = [
+            self._enrich_candidate_stats(cw, points, fps, frame_width, video_path, idx_cfg)
+            for cw in all_candidate_windows
+        ]
+
         # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
         candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
         if store_candidates:
             for i, cw in enumerate(all_candidate_windows):
-                stats = {
```

```
-                    "peak_velocity": cw["peak_velocity"],
-                    "mean_velocity": cw["mean_velocity"],
-                    "active_frame_count": cw["active_frame_count"],
-                    "track_id_count": cw["track_id_count"],
-                }
                 candidate_ids[i] = self.db.add_candidate_segment(
                     video_id, detector=self.name,
                     start_time=cw["start"], end_time=cw["end"],
                     chunk_index=cw["chunk_index"], confidence=min(1.0, cw["peak_velocity"]),
-                    stats=stats, detection_params=detection_params,
+                    stats=window_stats[i], detection_params=detection_params,
                 )

         if not all_candidate_windows:
             return []

         # --- Phase 3: AI provider handoff over padded candidate clips ---
+        # Which windows reach the handoff (base = all; fusion may apply Gate A/B). Persisted
+        # candidates above stay the full superset ? only the served play_segments are gated.
+        handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)
         handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
         os.makedirs(handoff_dir, exist_ok=True)
         logger.info("Phase 3: %s validation on %d candidate windows...",
-                    provider_name, len(all_candidate_windows))
+                    provider_name, len(handoff_indices))

         def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
             w_start = max(0, window["start"] - handoff_padding)
@@ -225,8 +255,8 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):

         ai_segments: List[dict] = []
         with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as executor:
-            futures = [executor.submit(process_candidate_window, wi, cw)
-                       for wi, cw in enumerate(all_candidate_windows)]
+            futures = [executor.submit(process_candidate_window, wi, all_candidate_windows[wi])
+                       for wi in handoff_indices]
             for future in futures:
                 ai_segments.extend(future.result())
         try:
```

9. user (2026-06-13T19:14:07.665Z)

```
diff --git a/backend/config/models.py b/backend/config/models.py
index bb56881..7f8191c 100644
--- a/backend/config/models.py
+++ b/backend/config/models.py
@@ -75,6 +75,39 @@ class WasbIndexConfig(BaseModel):
     min_free_gb: float = 0.0


+class FusionConfig(BaseModel):
+    """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
+
+    Every default reproduces control behaviour: the fusion segmenter persists the
+    shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']`` is
+    true ? the person-cue features, but it does NOT gate the served handoff unless a
+    threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court mask
+    is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-only);
+    supplied ? off-court persons (spectators / adjacent courts) are excluded.
+    """
+    # Which trajectory substrate seeds the windows (shuttle stays primary).
+    substrate: Literal["wasb", "tracknet"] = "wasb"
+    # Windowing velocity threshold for the fusion family (the engine reads
+    # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
```

```
+    # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
+    velocity_threshold: float = 0.02
+    # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
+    # enabled ? so the default path never imports torch / touches the GPU.
+    cue_flags: dict[str, bool] = Field(default_factory=dict)
+    # Served Gate A: drop windows whose shuttle net-crossings < this from the AI handoff.
+    # 0 = OFF (no gating; persisted candidates are always the full superset for offline re-
scoring).
+    min_crossings: int = Field(default=0, ge=0)
+    # Served Gate B: when the person cue is on, drop windows with median in-court players
+    # < this. 0 = OFF.
+    min_players: int = Field(default=0, ge=0)
+    # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
+    court_polygon: Optional[list[list[float]]] = None
+    # Net line for the person two-sided-motion split (person features only ? the shuttle
+    # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
+    net_axis: Literal["x", "y"] = "y"
+    net_position: float = Field(default=0.5, ge=0.0, le=1.0)
+
+
 class IndexingConfig(BaseModel):
     default_segmenter: str = "yolo_hybrid"
     use_gpu: bool = True
@@ -106,6 +139,7 @@ class IndexingConfig(BaseModel):

     tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
     wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
+    fusion: FusionConfig = Field(default_factory=FusionConfig)

     @field_validator("default_segmenter")
     @classmethod
diff --git a/backend/pipeline/detectors/person_detector.py
b/backend/pipeline/detectors/person_detector.py
index 38ec2b3..5b931a4 100644
--- a/backend/pipeline/detectors/person_detector.py
+++ b/backend/pipeline/detectors/person_detector.py
@@ -252,3 +252,50 @@ class PersonDetector:
         windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]

         return windows
+
+    def detections_per_frame(self, video_path: str, start_frame: int, end_frame: int,
+                             frame_skip: int = 3) -> Dict[str, Any]:
+        """Per-frame person detections over ``[start_frame, end_frame]`` of the ORIGINAL video.
+
+        Used by the fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2). Unlike
+        ``extract_action_windows_from_chunk`` (which detects ITS OWN windows and discards the
+        per-frame boxes), this returns the raw per-frame detections so the fusion features can
+        align them with the shuttle trajectory. It reads the source by **absolute frame index**
+        (seek + sequential read), NOT a re-extracted clip, so the returned frame indices match
+        the WASB trajectory's ``.frame`` exactly ? that alignment is what
+        ``fusion_features.shuttle_player_colocation`` / ``two_sided_motion`` rely on.
+
+        Returns ``{"frames": {frame_idx: [(track_id, bbox), ...]}, "fps", "frame_width",
+        "frame_height"}``. A fresh window-local ByteTrack tracker is used. ? cv2 frame-seek
+        accuracy varies by codec (it lands near the requested keyframe then decodes forward);
+        treat the indices as ±a few frames on real footage ? fine for window-level features.
+        """
+        self.lazy_load_model()
+        cap = cv2.VideoCapture(video_path)
+        if not cap.isOpened():
+            logger.error(f"Could not open {video_path}")
+            return {"frames": {}, "fps": 30.0, "frame_width": 0.0, "frame_height": 0.0}
+
+        fps = cap.get(cv2.CAP_PROP_FPS)
```

```
+        if fps <= 0:
+            fps = 30.0
+        frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
+        frame_height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
+        score_thresh = self.config.get("indexing", {}).get("detector_score_threshold", 0.5)
+        step = max(frame_skip, 1)
+        tracker = self.make_tracker(fps / step)
+
+        start_frame = max(0, int(start_frame))
+        frames: Dict[int, List[Tuple[int, Tuple[float, float, float, float]]]] = {}
+        cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
+        frame_idx = start_frame
+        while frame_idx <= end_frame:
+            ret, frame = cap.read()
+            if not ret:
+                break
+            if (frame_idx - start_frame) % step == 0:
+                frames[frame_idx] = self.detect_and_track(frame, tracker, score_thresh)
+            frame_idx += 1
+        cap.release()
+        return {"frames": frames, "fps": fps,
+                "frame_width": float(frame_width), "frame_height": float(frame_height)}
diff --git a/backend/pipeline/segmenters/__init__.py b/backend/pipeline/segmenters/__init__.py
index c003d1a..fc5de77 100644
--- a/backend/pipeline/segmenters/__init__.py
+++ b/backend/pipeline/segmenters/__init__.py
@@ -4,6 +4,7 @@ from backend.pipeline.segmenters.gemini import GeminiPlaySegmenter
 from backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
 from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
 from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
+from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter

 __all__ = [
     "segmenter_registry",
@@ -13,4 +14,5 @@ __all__ = [
     "HybridYoloSegmenter",
     "TrackNetHybridSegmenter",
     "WasbHybridSegmenter",
+    "FusionSegmenter",
 ]
```

10. assistant (2026-06-13T19:14:12.375Z)

Now I have the core diffs. Let me verify several specific claims from the finding. The most important one is claim #4 (the off-by-one / index alignment), and the handoff indexing. Let me look carefully at the refactored handoff loop.

The finding claims: "the handoff loop indexes candidate_ids[wi]/all_candidate_windows[wi] with full-list indices from handoff_indices ? no off-by-one". Let me verify the `process_candidate_window` body and how `wi` and `candidate_ids` are used.

11. user (2026-06-13T19:14:13.438Z)

```
218
219             if not all_candidate_windows:
220                 return []
221
222             # --- Phase 3: AI provider handoff over padded candidate clips ---
223             # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
224             # candidates above stay the full superset ? only the served play_segments are
gated.
225             handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)
226             handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
```

```
            os.makedirs(handoff_dir, exist_ok=True)
            logger.info("Phase 3: %s validation on %d candidate windows...",
                        provider_name, len(handoff_indices))

            def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
                w_start = max(0, window["start"] - handoff_padding)
                w_end = min(video_duration, window["end"] + handoff_padding)
                w_dur = w_end - w_start
                w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
                if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
                    return []
                prompt = sport_profile.get_prompt(chunk_duration=w_dur)
                segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
                                                     label=f"Handoff {wi+1}")
                valid = []
                for seg in segments:
                    try:
                        seg["start_time"] = float(seg["start_time"]) + w_start
                        seg["end_time"] = float(seg["end_time"]) + w_start
                        seg["_candidate_id"] = candidate_ids[wi]
                        valid.append(seg)
                    except (ValueError, TypeError, KeyError) as e:
                        logger.warning("Dropping segment with unparseable timestamps: %s ?
%s", seg, e)
                try:
                    os.remove(w_path)
                except OSError:
                    pass
                return valid

            ai_segments: List[dict] = []
            with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
                futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
                           for wi in handoff_indices]
                for future in futures:
                    ai_segments.extend(future.result())
            try:
                os.rmdir(handoff_dir)
            except OSError:
                pass
            logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))

            # --- Phase 4: DB population + candidate linking ---
            segments_data = []
            for segment in ai_segments:
                try:
                    start = float(segment["start_time"])
                    end = float(segment["end_time"])
                    dur = end - start
                    metadata = {
                        "shot_count": self._shot_count_from(segment, dur),
                        "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
                        "detector": f"{self.family}-hybrid-{model_name}",
                    }
                    segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
                    if segment_id:
                        candidate_id = segment.get("_candidate_id")
                        if candidate_id is not None:
                            self.db.link_candidate_to_segment(candidate_id, segment_id)
```

```
285                         segments_data.append({
286                             "id": segment_id, "start_time": start, "end_time": end,
287                             "duration": dur, "metadata": metadata,
288                         })
289                 except Exception as e:
290                     logger.warning("Skipping invalid segment struct: %s", e)
291
292             return segments_data
293
```

12. assistant (2026-06-13T19:14:24.127Z)

The handoff indexing is correct. `wi` comes from `handoff_indices` which are indices into the full `all_candidate_windows` list, and `candidate_ids[wi]` and `all_candidate_windows[wi]` both index the full lists with the same `wi`. So `_candidate_id = candidate_ids[wi]` correctly links the AI segment back to the right persisted candidate. No off-by-one. The finding's claim #4 (well, the structural integrity claim) holds.

Now let me verify the byte-identical equivalence claim. The base `_enrich_candidate_stats` returns exactly the 4 keys that the old inline `stats` dict had. Confirmed in the diff: old inline dict had `peak_velocity`, `mean_velocity`, `active_frame_count`, `track_id_count` ? and the new base method returns exactly those 4. So persisted stats are byte-identical for the twins. Good.

Now let me verify the config-related claims by actually running Python, since the environment claims pydantic is present. Let me check claims #1, #2, #3, #4 (config), #5 (registry), #6 (probe).

13. user (2026-06-13T19:14:25.476Z)

```
pydantic 2.13.4
```

14. assistant (2026-06-13T19:14:30.232Z)

pydantic 2.x is present. Now let me check the `Literal`/`Optional` imports in models.py and the AppConfig dict-shim. Let me read the relevant parts of models.py.

15. user (2026-06-13T19:14:31.101Z)

```
1       """Pydantic models for the application configuration.
2
3       This module defines the canonical, validated, typed configuration for the
4       Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6       The models support graceful migration from old config.json files via extra="allow".
7       """
8
9       from __future__ import annotations
10
11      from typing import Any, List, Literal, Optional
12
13      from pydantic import BaseModel, Field, field_validator, model_validator
14
15      Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18      class ValidationRelevanceConfig(BaseModel):
19          court_green_lower: list[int] = Field(default=[35, 40, 40])
20          court_green_upper: list[int] = Field(default=[85, 255, 255])
```

16. assistant (2026-06-13T19:14:37.807Z)

Important detail: the docstring says "The models support graceful migration from old config.json files via `extra="allow"`." This contradicts the finding's claim #4, which states "FusionConfig

has no extra='allow' so default extra='ignore' applies ? unknown keys are silently dropped". Let me check whether the base models actually set `extra="allow"` somewhere (a base class or model_config). If `extra="allow"` is set, then unknown keys would NOT be dropped ? they'd be kept. Either way they don't crash, but the finding's specific mechanism claim ("extra='ignore' applies, unknown keys silently dropped") could be wrong. Let me check.

17. user (2026-06-13T19:14:38.756Z)

```
6:The models support graceful migration from old config.json files via extra="allow".
59:    because nested config sections defaulted to extra='ignore'.
233:    # Allow extra keys during transition so existing config.json files do not break.
234:    model_config = {
235:        "extra": "allow",
```

18. assistant (2026-06-13T19:14:42.341Z)

Let me look at the context around lines 55-60 and 230-240 to see which classes have `extra="allow"` vs `extra="ignore"`.

19. user (2026-06-13T19:14:42.798Z)

```
50          # 0 = disabled.
51          min_free_gb: float = 0.0
52
53
54      class WasbIndexConfig(BaseModel):
55          """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57          Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58          these knobs actually take effect ? previously this whole block was silently dropped
59          because nested config sections defaulted to extra='ignore'.
60          """
61          wsl_distro: str = "Ubuntu"
62          repo_dir: str = "~/models/WASB-SBDT"
63          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64          conda_env: str = "wasb"
```

20. user (2026-06-13T19:14:46.231Z)

```
225         ai_model: str = "gemini-flash-latest"
226
227         validation: ValidationConfig = Field(default_factory=ValidationConfig)
228         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
229         output: OutputConfig = Field(default_factory=OutputConfig)
230         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
231         security: SecurityConfig = Field(default_factory=SecurityConfig)
232
233         # Allow extra keys during transition so existing config.json files do not break.
234         model_config = {
235             "extra": "allow",
236             "validate_assignment": True,
237         }
238
239         # Temporary migration shim for legacy dict-style access (config.get("section",
{}).get(...)).
240         # Will be removed once all call sites (validators, segmenters, etc.) move to typed
241         # attribute access / the typed config. A nested *section* is returned as a plain
dict
242         # via model_dump, so legacy chained access keeps working ? returning the Pydantic
243         # section model instead breaks it (a model has no .get), which bit every validator
on
244         # real runs. Scalar leaves are returned as-is.
```

21. assistant (2026-06-13T19:14:56.861Z)

So `extra="allow"` is only on the top-level `AppConfig` (line 234), not on `FusionConfig` or the other nested sections. The comment at line 58-59 confirms nested sections default to `extra="ignore"`. So the finding's claim #4 mechanism is correct: `FusionConfig` (a nested `BaseModel`) uses the default `extra="ignore"`, so unknown fusion keys are silently dropped without crashing.

But wait ? the finding also says "both directly and via AppConfig(indexing={'fusion':{'unknown_key':123}})". The path is `AppConfig` ? `indexing` (IndexingConfig) ? `fusion` (FusionConfig). The `fusion` field is typed as `FusionConfig`, so even though `AppConfig` has `extra="allow"`, the `fusion` sub-object is validated by `FusionConfig` which ignores extras. So unknown keys inside `fusion` get dropped. Correct.

Now let me empirically verify all the config claims at once with a Python script, plus the registry and probe claims.

22. user (2026-06-13T19:15:02.915Z)

```
DEFAULTS: wasb {} 0 0 None 0.02 y 0.5
OK: bad substrate rejected
OK: bad net_axis rejected
OK: negative min_crossings rejected
OK: net_position>1 rejected
parsed poly+cue: [[0.1, 0.1], [0.9, 0.1], [0.9, 0.9]] {'person': True}
```

23. assistant (2026-06-13T19:15:06.076Z)

Claims 1 and 2 verified empirically. Now claims 3, 4, 5, 6.

24. user (2026-06-13T19:15:10.769Z)

```
idx type: dict
fusion in idx: True fusion type: dict
substrate via shim: wasb | cue_flags: {} | min_crossings: 0 | court_polygon: None
unknown key dropped, substrate kept: tracknet | has unknown_key attr: False
model_dump fusion: {'substrate': 'tracknet', 'velocity_threshold': 0.02, 'cue_flags': {},
'min_crossings': 0, 'min_players': 0, 'court_polygon': None, 'net_axis': 'y', 'net_position':
0.5}
```

25. assistant (2026-06-13T19:15:15.706Z)

Claims 3 and 4 verified. The dict-shim `cfg.get('indexing')` returns a plain nested dict with `fusion` itself a dict, and chained `.get()` resolves. Unknown fusion keys are silently dropped (no ValidationError) while known keys are kept.

Now claims 5 (no name clash, registry list) and 6 (registry probe `FusionSegmenter(None, {})` safe).

26. user (2026-06-13T19:15:25.662Z)

```
registry api: ['get', 'list_available', 'register']
```

27. user (2026-06-13T19:15:38.901Z)

```
registered names: ['fusion_hybrid', 'gemini', 'motion', 'tracknet_hybrid', 'wasb_hybrid',
'yolo_hybrid']
name: fusion_hybrid
description ok: True
config: {} | _person: None | family: fusion
PASS probe
```

28. assistant (2026-06-13T19:15:43.299Z)

All six claims verified empirically. The registry contains `fusion_hybrid` distinct from `wasb_hybrid`/`tracknet_hybrid`/`yolo_hybrid`/`gemini`/`motion` ? no name clash.

`FusionSegmenter(None, {})` constructs safely (config={}, _person=None, name is a plain string property).

Now let me double check the equivalence test claim and run the actual test suites referenced to confirm they pass. The finding references `tests/test_fusion_hybrid.py`, `tests/test_fusion_features.py`, `tests/test_config.py`, `tests/test_trajectory_hybrid_equivalence.py`. Let me run those.

29. user (2026-06-13T19:15:52.842Z)

```
.....................................                                    [100%]
42 passed in 6.37s
```

30. assistant (2026-06-13T19:16:01.997Z)

All 42 tests pass across the four referenced suites. Let me also verify the equivalence test actually asserts byte-identical stats/behavior for the twins (to confirm the refactor preserved them), and double-check the `_select_for_handoff` base returns the full index range so no windows are silently dropped.

31. user (2026-06-13T19:16:03.224Z)

```
warning: in the working copy of 'tests/test_trajectory_hybrid_equivalence.py', LF will be
replaced by CRLF the next time Git touches it
diff --git a/tests/test_trajectory_hybrid_equivalence.py
b/tests/test_trajectory_hybrid_equivalence.py
index 495a0cd..dfe58af 100644
--- a/tests/test_trajectory_hybrid_equivalence.py
+++ b/tests/test_trajectory_hybrid_equivalence.py
@@ -100,3 +100,25 @@ def test_base_is_not_registered():
     # The base engine itself must not appear as a selectable segmenter.
     assert not any(isinstance(cls, type) and cls is TrajectoryHybridSegmenter
                    for cls in available.values())
+
+
+@pytest.mark.parametrize("cls,family,detector", TWINS)
+def test_persisted_stats_are_exactly_the_four_motion_keys(cls, family, detector):
+    """The fusion-P2 enrichment seam must leave the twins' persisted stats byte-identical:
+    the base _enrich_candidate_stats is identity, so NO net_crossings / person keys leak in."""
+    db, _res = _run(cls, [{"start_time": 0.5, "end_time": 3.0}])
+    _, kwargs = db.add_candidate_segment.call_args
+    assert set(kwargs["stats"]) == {
+        "peak_velocity", "mean_velocity", "active_frame_count", "track_id_count"}
+
+
+@pytest.mark.parametrize("cls,family,detector", TWINS)
+def test_base_fusion_hooks_are_identity(cls, family, detector):
+    """Directly pin the two fusion seams on the base: stats = the 4 motion keys verbatim,
+    and every window is selected for handoff (no gating)."""
+    seg = cls(MagicMock(), {"indexing": {}})
+    cw = _window()
+    stats = seg._enrich_candidate_stats(cw, [], 30.0, 1280.0, "v.mp4", {})
+    assert stats == {"peak_velocity": 0.4, "mean_velocity": 0.2,
+                     "active_frame_count": 30, "track_id_count": 1}
+    assert seg._select_for_handoff([cw, cw, cw], [stats, stats, stats], {}) == [0, 1, 2]
```

32. assistant (2026-06-13T19:16:14.728Z)

The equivalence test directly pins both seams as identity on the twins (WASB/TrackNet hybrid): persisted stats are exactly the 4 motion keys, and `_select_for_handoff` returns the full index range `[0,1,2]`. This confirms the refactor preserves the twins byte-identical.

Let me do one final adversarial check on the broader test suite to make sure nothing else regressed, focusing on the segmenter/registry/person slice, and confirm the equivalence test

isn't somehow skipped.

33. user (2026-06-13T19:16:23.837Z)

```
.........................................                                [100%]
44 passed in 6.00s
```

34. assistant (2026-06-13T19:16:39.891Z)

All checks pass. Let me consolidate my adversarial verification.

I attempted to refute each of the six claims in the finding and could not. Every claim holds against the actual code, verified both by reading and by live Python execution:

1. **Defaults reproduce control** ? `FusionConfig()` yields `substrate='wasb'`, `cue_flags={}`, `min_crossings=0`, `min_players=0`, `court_polygon=None`, `velocity_threshold=0.02` (verified live). Base `_select_for_handoff` returns `list(range(len(windows)))` and base `_enrich_candidate_stats` returns exactly the 4 motion keys (fusion_hybrid.py:114, trajectory_hybrid.py base seams). When person is OFF, `_enrich_candidate_stats` adds only `net_crossings` (fusion_hybrid.py:78,80-83).

2. **Pydantic types valid** ? instantiation works; `Literal['wasb','tracknet']`, `Literal['x','y']`, `ge=0`, `ge=0.0/le=1.0`, `Optional[list[list[float]]]`, `dict[str,bool]` all validate/reject as claimed (verified live).

3. **Dict-shim exposes fusion** ? `AppConfig().get('indexing')` returns a plain nested dict with `fusion` itself a dict; chained `.get('substrate'/'cue_flags'/'min_crossings'/'court_polygon')` all resolve (verified live).

4. **Unknown fusion keys don't crash** ? `FusionConfig` (nested model) uses default `extra="ignore"`; only top-level `AppConfig` sets `extra="allow"` (models.py:234). `AppConfig(indexing={'fusion':{'unknown_key':123}})` drops the unknown key silently, no `ValidationError` (verified live). The finding's mechanism description is accurate.

5. **No name clash** ? registry contains `fusion_hybrid`, `gemini`, `motion`, `tracknet_hybrid`, `wasb_hybrid`, `yolo_hybrid`; `fusion_hybrid` is distinct, `family='fusion'` is a fresh config section (verified live).

6. **Registry probe safe** ? `FusionSegmenter(None, {})` constructs (config={}, _person=None), `.name` and `.description` resolve (verified live).

The off-by-one concern I specifically hunted for is clean: the handoff loop indexes `all_candidate_windows[wi]` and `candidate_ids[wi]` with full-list indices from `handoff_indices` (trajectory_hybrid.py:246,258), so `_candidate_id` linkage is correct. The equivalence tests pin the twins as byte-identical. All 44 relevant tests pass.

This finding is an "info"-severity confirmation that the integration is sound, not a bug report. It is accurate. There is no high/medium integration bug to confirm or refute ? the finding correctly asserts none exists.

35. user (2026-06-13T19:16:55.737Z)

Structured output provided successfully

36. assistant (2026-06-13T19:17:02.717Z)

I have completed the adversarial verification. The finding is **real (info severity)** ? it correctly confirms that the fusion-P2 integration is sound with no high/medium integration bugs.

I attempted to refute all six claims and could not. I verified each both by reading the actual code and by running live Python:

- **Defaults reproduce control**, **pydantic types valid**, **dict-shim exposes fusion**, **unknown keys don't crash**, **no name clash**, **registry probe safe** ? all confirmed

empirically.
- The off-by-one risk I specifically hunted for (the handoff-index refactor in `trajectory_hybrid.py:246,258`) is clean: `wi` indexes both `all_candidate_windows[wi]` and `candidate_ids[wi]` with the same full-list index, so candidate?segment linkage is correct.
- The twins remain byte-identical (base `_enrich_candidate_stats` returns exactly the 4 motion keys that the old inline dict had).
- 42 tests pass across the four referenced suites; 44 including the person-detector suite.

The only caveats are owner-side real-video limitations already documented in code comments (cv2 codec-dependent frame-seek accuracy; the person/GPU path is mock-tested only) ? documented limitations, not bugs.